and Agha 2007; Tasharofi et al. 2012]. The obtained reduction can, however, vary significantly depending on several factors, e.g., the order in which processes are explored at each point of scheduling. For a particular implementation of DPOR, up to an order of magnitude of difference in the number of explored interleavings has been observed when different strategies are used [Tasharofi et al. 2012]. Heuristics for choosing which next process to explore have been investigated without conclusive results [Lauterburg et al. 2010]. It has also been shown that for specific communication models, specializations of DPOR algorithms, obtained by carefully defining when execution steps are in conflict (for example, exploiting the transitivity of the dependency relation in actor systems [Tasharofi et al. 2012]), can achieve better reduction.

Let us explain one fundamental reason for the above variation in obtained reduction. In DPOR, the combination of persistent set and sleep set techniques guarantees to explore at least one complete interleaving in each Mazurkiewicz trace. Moreover, it has already been proven that the use of sleep sets is sufficient to prevent the complete exploration of two different but equivalent interleavings [Godefroid et al. 1995]. At first sight, this seems to imply that sleep sets can give optimal reduction. What it actually implies, however, is that when the algorithm initiates the exploration of an interleaving which is equivalent to an already explored one, the exploration will begin but it will be blocked sooner or later by the sleep sets in what we call a *sleep set blocked* exploration. When only sleep sets are used for reduction, the exploration effort will include an arbitrary number of sleep set blocked explorations. It is here where persistent sets enter the picture, and limit the number of initiated explorations. Computation of smaller persistent sets leads to fewer sleep set blocked explorations. However, as we show in this article, persistent sets are not powerful enough to completely prevent sleep set blocked exploration.

Another reason for the observed non-optimal reduction is that most published algorithms define conflicts as occurring when two statements access the same shared object, and at least one of them modifies the object. However, in many cases more reduction can be achieved by using a more refined definition of conflicts. For instance, two writes to a shared variable need not conflict if they write the same value. As another example, a send statement and a receive statement that access the same message queue need not be in conflict, unless the two statements concern the same message. Thus, a reduction technique would benefit from a refined definition of dependencies, which considers not only the accessed shared object, but also the actual effect of each program statement in a given execution. Such a refined definition would be particularly beneficial for programs that employ message passing communication.

In view of these variations, a fundamental challenge is to develop an *optimal* DPOR algorithm that: (i) always explores the minimum number of interleavings, regardless of scheduling decisions, (ii) can be efficiently implemented, and (iii) is applicable to a variety of computation models, including concurrency via message passing, communication via shared variables protected by locks, or in more general forms of interprocess communication. Such an optimal DPOR algorithm could be standardly implemented in stateless model checkers and test generation tools for concurrent systems. It would also attain the maximal reduction that can be achieved using information about conflicts between processes, implying that any further reductions must be obtained by other means, e.g., by symbolic techniques, SAT-based approaches such as the recently proposed SATCheck technique [Demsky and Lam 2015], or by techniques that take alternative notions of causality into account such as Maximal Causality Reduction [Huang 2015].

Contributions. In this article, we present a fundamentally new DPOR technique, which is based on a new theoretical foundation for partial order reduction, in which persistent sets are replaced by a novel class of sets, called *source sets*. Source sets subsume persistent sets (i.e., any persistent set is also a source set), and source sets are often smaller than persistent sets. Moreover, source sets